

Secure Machine Learning in the Cloud Using Private-Key Deconvolution

ABSTRACT

Cloud-based machine learning services (CMLS) enable organizations to take advantage of advanced models that are pre-trained on large quantities of data. The main shortcoming of using these services, however, is the difficulty of keeping the transmitted data private and secure. Asymmetric encryption requires the data to be decrypted in the cloud, while Homomorphic encryption is often too slow and difficult to implement. We propose Encoding via Private-Key Deconvolution (EPKD), a deconvolution-based encryption framework that offers a non-trivial level of security at a fraction of the computational overhead required by fully-secure Homomorphic encryption-based techniques. Extensive evaluation on multiple image datasets demonstrates EPKD's ability to achieve near-perfect classification performance when the output vector of the CMLS is sufficiently large. Additionally, we provide a comprehensive analysis of the robustness of our approach.

CCS CONCEPTS

• **Security and privacy** → **Software and application security**;
• **Computing methodologies** → **Neural networks**; **Learning latent representations**.

KEYWORDS

Neural Networks, Encryption, Cloud-based Service, Confidentiality

ACM Reference Format:

. 2018. Secure Machine Learning in the Cloud Using Private-Key Deconvolution. In *Proceedings of* . ACM, New York, NY, USA, 10 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 INTRODUCTION

Recent advances in Cloud-based Machine Learning Services (CMLS) capabilities have made it possible for individuals to access state-of-the-art algorithms that until recently were only accessible to few. These capabilities, however, come with two major risks: *a*) the leakage of sensitive data as it's being sent to and/or processed by the cloud, and; *b*) the ability of third parties to gain insight into the organization's data by analyzing the output of the ML model in the cloud.

Organizations that transmit their data to a CMLS have two options to secure their data: *a*) Send the data in encrypted form to the cloud, where it will be decrypted, analyzed and then sent back in encrypted form (e.g., using asymmetric encryption); *b*) Employ

Homomorphic Encryption (HE) techniques that enable the cloud to process the data in its encrypted form. The first option—decryption in the cloud—enables the cloud provider unfettered access to the organization's private data, and is unacceptable in many cases. HE keeps the data private, even when it is being processed in the cloud, but it is slow and puts various limitations on the operations that can be performed on the cloud side given known HE schemes.

In this study we propose Encoding via Private-Key Deconvolution (EPKD), an approach that provides the main advantage of Homomorphic encryption—not sharing the unencrypted data with any party, including the CMLS—but with a computational overhead that is smaller by orders of magnitude. Our approach uses a randomly-initialized deconvolutional architecture to encrypt the input, which is then sent to the CMLS. The CMLS processes the input in the same manner as it would any unencrypted data, and produces a classification vector. Finally, a small “translator” neural architecture within the organization receives the classification vector produced by the cloud and combines it with an embedding of the original image to predict the classification the cloud service would have given to the original image.

EPKD has several important advantages compared to existing methods. First, the use of randomly-initialized deconvolutional networks creates encrypted images that are both incomprehensible to human observers and whose classification in the cloud does not disclose their original label. Secondly, as the deconvolutional network is randomly initialized and not trained, replacing the “key” (i.e., network weights values) of our encryption mechanism is computationally inexpensive, as we only need to retrain our translator architecture. Thirdly, our approach does not require any coordination with the CMLS, since the latter processes our encrypted images just as it would any plaintext image.

Our contributions in this study are as follows:

- We present EPKD, a deconvolution-based encryption approach that has many of the advantages of HE at a fraction of the computational cost.
- Extensive evaluation shows our approach can reach up to 99.9% of the performance of the CMLS when the data has fewer labels than the cloud. Additionally, it doesn't matter if the data and the CMLS have any overlap in the labels.
- We perform a comprehensive analysis of the robustness of our approach to attacks, and provide our dataset to enable reproducibility of our results.

2 RELATED WORK

2.1 Encryption-based solutions

Solutions that provide privacy-preserving Machine Learning (ML) in the cloud can be divided into two groups: one in which the cloud-based service provider has access to the plaintext, and one in which it does not. The former group of solutions utilizes common public/private key settings (e.g., RSA [27]) and is in widespread use. In [24], the authors present a multi-step framework for the

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted by ACM, provided that the copies are not made for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2018 Association for Computing Machinery.
ACM ISBN 978-x-xxxx-xxxx-x/YY/MM...\$15.00
<https://doi.org/XXXXXXX.XXXXXXX>

exchange of public keys. A semi-honest model—a scenario where the cloud provider is not compromised—is also assumed. Similar approaches have also been proposed by [24, 26]. In all of these solutions, however, the cloud has full access to the plaintext, a fact that could be highly problematic if the plaintext consists of sensitive information (say medical records).

The latter group of solutions can be divided into hardware and software-based solutions. In hardware-based solutions, multiple studies [18, 19, 35] utilize the SGX architecture which possess a secure enclave for the processing of sensitive information. In [18], for example, the authors demonstrate how to effectively run neural architectures on SGX. This approach, while effective, requires dedicated software both by the client and the service provider, as well as adaptation of the ML model.

Software-based encryption solutions that withhold the plaintext from the CMLS, though diverse, mostly revolve around the use of HE. In [25] the authors propose two Fully Homomorphic Encryption (FHE) schemes for privacy-preserving deep learning. In [13, 14], the authors demonstrated that it is feasible to train neural networks using encrypted data, while in [11] the authors detailed a method to convert learned neural networks to neural networks that can be applied to encrypted data (using leveled HE). While they enable full protection of one’s data (assuming standard cryptographic hardness assumptions), HE-based solutions are slow, require large amount of memory and prior knowledge of the type of queries, so that the appropriate HE scheme can be selected to encrypt the raw data [25].

2.2 Instance Encoding for Secure Machine Learning

Another field of research strongly related to ours is the one termed by [3] as private instance encoding. In this approach, the data is encoded in a way that prevents the reconstruction of the plaintext data. The encoded data, however, is still meaningful enough that it can be used to train ML models. Two recent studies that proposed such solutions are NueraCrypt [38] and InstaHide [17]. The former approach uses randomly-initialized convolutions for image transformation, combined with local and global mixing of images. The latter uses linear pixel blending to merge multiple images, then randomly flips the pixel signs.

Despite their different approaches, both NueraCrypt and InstaHide were broken. In [4] and [3], the authors demonstrated how images can be reconstructed from their encoded forms. Moreover, in [3], the authors claim that no useful instance encoding scheme is secure against *distinguishing attacks*—the detection of similar samples, even in their encoded state.

While our approach (EPKD) is reminiscent of the abovementioned instance encoding solutions, critical differences exist. First and foremost, our encoded samples do not include a label, thus denying an important source of information. Secondly, we use vanilla deconvolutional architectures for our encoding, without the “add-ons” used in previous works. This makes our approach more robust. Thirdly, we show that our encodings are not sufficiently informative to meet the base assumptions detailed in [3]. Thus, we challenge the claim that “no useful instance encoding scheme is secure against distinguishing attacks”, basically, by providing the

cloud with much less information and more limited access than previous suggestions. Finally, EPKD’s ability to easily generate new (random) keys means the encrypted data does not have one consistent representation. The limited number of samples per key further limits the information available to attackers.

2.3 Model Extraction Attacks

Model extraction attacks aim to steal the functionality or the parameters of a remote machine learning model. Given the amount of time, data, and computing resources needed to train large-scale neural architectures, it is understandable that these models are often a target for attacks of this kind.

White-box ML models are highly vulnerable to model extraction attacks, as well as adversarial attacks [2, 34] and model inversion attacks [6, 10], whose aim is to reveal confidential information that was used to train the model thorough aggregated statistics.

Model extraction attacks on black-box models are naturally more difficult, and can be roughly divided into two sub-types: *task accuracy extraction* and *fidelity extraction*. Task accuracy extraction attacks focus on extracting a model that performs well on the underlying task, while fidelity extraction attacks also aim to reproduce the predictions made by the black-box model.

A more challenging sub-field of fidelity extraction attacks aims to replicate the weights of the black-box models with high precision. The main difficulty in this high-precision extraction task is the need to accurately model a large number (possibly millions) of floating-point values. One approach to handle floating point imprecision was presented in [20], where the authors used finite differences to estimate the gradient. The work presented in [30] enables high-precision extraction, but required tens to hundreds of millions of queries for relatively small networks. A recent work by [7] proposed a more efficient way to reverse-engineer the weights of a neural network, but the approach can only be applied to neural networks with a single hidden layer.

The most advanced work to date in the field of high-precision model extraction was presented by [5]. This study builds on techniques from the cryptanalysis domain: by identifying the critical points of individual neurons, the authors sequentially isolate the weights of each layer in the neural network. To make their attack more efficient, the authors employ multiple techniques, including binary search, graph clustering, and mixed integer linear programming [21].

3 THE PROPOSED METHOD

3.1 Overview

Our proposed approach is presented in Figure 1. EPKD consists of three components: *a)* an *encoder*, that creates an embedding of the original image; *b)* a *generative model*, that receives the embedding as input and creates the encrypted image, and; *c)* an *internal inference network*, which receives the classification vector for the encrypted image from the cloud and infers the labels of the plaintext images. **The Encoder.** The goal of this component is to create a meaningful condensed representation of the original (unencrypted) image. Multiple convolutional architectures would be suitable for this task. In our experiments we used pre-trained and well-known image classification architectures (e.g., ResNet50, ResNet50V2, ResNet101)

whose softmax layers were removed to create our embeddings.

The Generative Model. The goal of this component is to create an encrypted version of the original image. We use a deconvolutional architecture that receives the embedding generated by the encoder and outputs a new image. The new image is encrypted because it is generated by a *network whose weights are randomly initialized*. This set of randomly-initialized weights serves as EPKD’s encryption key. Next, the newly-generated (i.e., encrypted) image is sent to the CMLS. The service processes the image as it would any unencrypted input, producing a classification vector. This vector is then sent back to the secure organizational network. It is critical to note that *only encrypted images ever leave the organizational network*, both during the training of model (on which we elaborate later in this section) and when it is applied on new data.

The Internal Inference Network (IIN). The dense neural net infers the label of the plaintext image. It receives as input the embedding of the original image (the Encoder’s output) and the classification vector produced by the CMLS. The IIN then *infers the true label of the original image*. This network can be trained using a small number training inputs, *requiring 4-20 minutes on a standard laptop*. It is important to note that the input size of the IIN is not dependent on those of the generative model. As a result, we can increase the size of the key without modifying the IIN. The input size of the IIN is dependent solely on the size of the vectors produced by the Encoder, and the CMLS output vector.

3.2 Formal Model and Definitions

We define the syntax and properties we want our system to satisfy. We let $\mathcal{M}$ be the space of images we want to handle.

- $\text{Enc}(\text{key}, m) \rightarrow (\widehat{m}, \sigma)$: A randomized procedure that gets as input a key key and an image $m \in \mathcal{M}$. It outputs an encoding $\widehat{m}$ and a private state σ .¹
- $\text{CMLS}(\widehat{m}) \rightarrow \widehat{o}$: A randomized procedure that gets as input an encoding $\widehat{m}$. It outputs $\widehat{o}$, an encoded classification vector.
- $\text{TrainIIN}^{\text{Enc}(\cdot, \cdot), \text{CMLS}(\cdot)}() \rightarrow z$: A deterministic procedure that can query the Enc and CMLS functions and produces a string z .
- $\text{IIN}(z, \widehat{o}, \sigma) \rightarrow o$: A deterministic procedure that gets as input the encoded classification vector $\widehat{o}$ and a private state σ . It outputs o , a classification vector.

Correctness. We want the IIN to output a “correct” classification vector. Namely, fix $f: \mathcal{M} \rightarrow \mathcal{O}$, a mapping from the images space $\mathcal{M}$ to the classification vectors space $\mathcal{O}$. We require that for any $m \in \mathcal{M}$ and every key key , letting $\text{TrainIIN}^{\text{Enc}(\text{key}, \cdot), \text{CMLS}(\cdot)}() \rightarrow z$ and $\text{Enc}(\text{key}, m) \rightarrow (\widehat{m}, \sigma)$, it holds that

$$\Pr[\text{IIN}(z, \text{CMLS}(\widehat{m}), \sigma) = f(m)] = 1 - \epsilon,$$

where $\epsilon > 0$ is a small constant (say 0.1) and the probability is over the randomness of TrainIIN , Enc and CMLS .

¹In our system, the Enc function is implemented via an encoding which is then fed into the generative model.

Security. The goal of a system satisfying the above syntax is to hide the contents of m from the CMLS phase. We capture this requirement via the following definition:

Definition 3.1 ($(\ell, \delta, \mathcal{D})$ -indistinguishability). Fix input images $\{m_{i,b}\}$ for $i \in \{1, \dots, \ell\}$ and $b \in \{0, 1\}$, each from $\mathcal{M}$. Fix a class of distinguishers $\mathcal{D}$. Fix $D \in \mathcal{D}$ and let $(\widehat{m}_{i,b}, \sigma_{i,b}) \leftarrow \text{Enc}(\text{key}, m_{i,b})$ for $i \in \{1, \dots, \ell\}$ and $b \in \{0, 1\}$. Then, for uniformly random $b \leftarrow \{0, 1\}$,

$$|\Pr[D(\{\widehat{m}_{i,b}\}_{i \in \{1, \dots, \ell\}}) = b] - 1/2| \leq \delta.$$

In words, the above definition says that no distinguisher should be able to distinguish (with probability better than δ) a sequence of encodings of images from any other sequence of encodings of images.

FHE-based solutions. A system satisfying the above definitions can be constructed using a Fully Homomorphic Encryption scheme (FHE). Indeed, if we merely encrypt every image with an FHE and require the model to perform classification homomorphically, we can later decrypt the encrypted classification. This clearly satisfied both correctness and security assuming the FHE scheme is secure. Unfortunately, this solution has several drawbacks. First, FHE is still rather expensive computationally and so it is challenging to deploy such a system in scale. Second, requiring the CMLS to perform FHE computations usually requires changing the underlying NN architecture. Due to the above, while FHE-based solutions give strong security guarantees, they are still somewhat far from being useful as a practical solution for generic deployed system.

Looking forward, our work suggests a heuristic middle ground between solutions without any security guarantees and fully-secure solutions such as the ones based on FHE. Specifically, we design, implement, and experiment with a new system that can be implemented and deployed rather easily in scale and importantly does not require modifying the CMLS to accommodate our system. We experiment with the system and show that it satisfies statistical indistinguishability properties (as in Definition 3.1) for a limited class of attacks. Further extending the class of attacks for which our system resists or formally proving security of the current system for restricted attacks is an effort left for future work.

3.3 The Training Phase

The only component of EPKD that requires training is the IIN. To train this component in a fully secure manner, the organization requires a small dataset of images D whose real labels are known. These images can then be encrypted, sent to the cloud, and have their classification vectors retrieved. We then use the embedding of each image (the output of the Encoder) and the CMLS’s classification vector to train the IIN to predict the true label of the original image.

We consider the requirement that organizations have a small labeled dataset to be reasonable, since most organizations have proprietary knowledge they wish to protect and therefore able to leverage for EPKD’s training. If such a dataset is not available, however, it is also possible to use publicly-available images to train the inference network. We explore such a scenario in use-case 3 of Section 4.2.

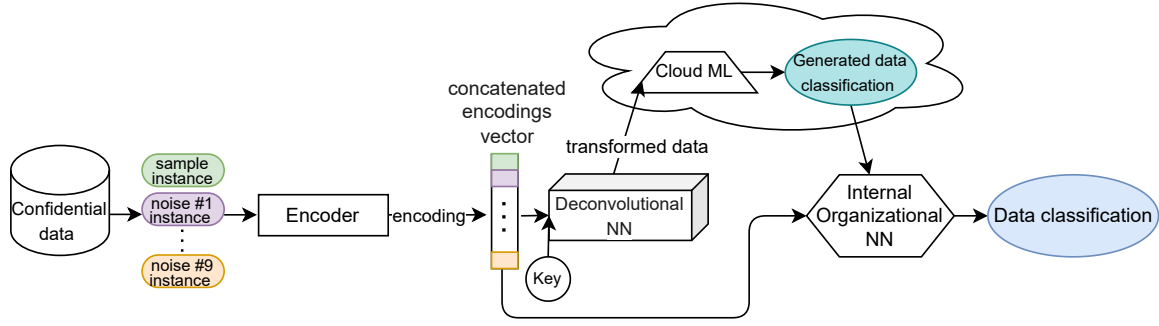


Figure 1: The EPKD architecture. Plaintext images are first transformed into an embedding, then expanded back to their original dimensions in encrypted form using a deconvolutional network. The deconvolutional network’s weights are initialized randomly, and are in fact the encryption key. The encrypted images are sent to the cloud, and a classification vector is obtained. The Internal Organizational NN translates these classifications into the final (real) classification.

3.4 Rationale

Our work builds upon two research areas: membership inference attacks [31] and distillation [15]. In the former, an attacker can learn which samples were used to train a black-box model by analyzing its output vector, thus showing that one can derive meaningful insights regarding the neural network’s inner workings from its output vector. In the latter, a single neural network is used to learn the decision process of an entire ensemble, and is thus able to achieve similar performance at a fraction of the computational resources. We combine these ideas in EPKD: we analyze the output vectors for encrypted images, and train a small neural network to distill the performance of the cloud-based model, although for predicting possibly different labels than the cloud’s.

4 EXPERIMENTS

4.1 Experimental setup

We used four datasets in our experiments: *a)* two variants of ImageNet [9]: ILSVRC2010 and ILSVRC2012; *b)* CIFAR-100, and; *c)* CIFAR-10 [22]. For our CMLS, we used the following pre-trained architectures: InceptionV3 [33], Xception [8], and VGG16 [32]. We trained the two former architecture on ILSVRC2012 (i.e., ImageNet), and the latter on CIFAR10 and CIFAR100. For the Encoder, we used pre-trained ResNet50, ResNet101 [12], and ResNet50V2 [29] architectures, all trained on the ILSVRC2012 training set. Please note that the same pre-trained architecture is *never used simultaneously in the Encoder and as the CMLS in our experiments*.

In our experiments, we use ten images for each encrypted image—the one we wish to protect, and nine other randomly-sampled images (from non-participating ImageNet labels). The Encoder outputs a vector of size 2,048 for each image. We then concatenate the vectors in a random order, so the position of the embedding of the image we wish to protect is not fixed, and use them as input for the Generative Model. For the latter, we use the DCGAN [28] generator, whose output dimensions are 256x256x3. The DCGAN weights are initialized using The Glorot uniform initializer. For the IIN, we use a dense net with a single hidden layer, followed by a softmax output layer. The IIN’s input vector is $10 \cdot 2,048 + 9 \cdot |C| + |V|$, where

Internal Inference				Cloud Performance	
Real Labels		Cloud Class.		Un-encrypted images	
Top-1	Top-5	Top-1	Top-5	Top-1	Top-5
17%	64.6%	17%	64.5%	93.3%	99.8%
(0.01)	(0.019)	(0.008)	(0.019)	(0.009)	(0.002)

Table 1: The evaluation results for our preliminary use-case, where the IIN is trained using only the prediction vectors of the CMLS on the encrypted images. “Internal Inference” refers to the performance of the IIN, and “cloud performance” presents the results on the plaintext images (i.e., an upper bound on performance). We present the ability of the IIN to predict both the ground-truth labels, and the labels produced by the CMLS.

2,048 is the embedding size of a single image, C is the number of original labels, and V is the CMLS’s output. To train the IIN, we used batch normalization, dropout, and Adam optimization. We used a learning rate of 0.0001 with exponential decay, and trained the network up to 40 epochs, with early stopping.

4.2 Evaluation Results

We evaluate our approach using two metrics—top-1 and top-5 accuracy—which are the standard in image classification tasks. We calculate these metrics with respect both to the ground truth (i.e., true labels) and to the cloud’s classification of the unencrypted image (i.e., we measure our ability to infer the cloud’s classifications).

Preliminary analysis: the usefulness of the output vectors. A key hypothesis of EPKD is that the CMLS’s output is informative when applied to our encrypted images. To test this, we run an experiment *where we only use the output vectors to train the IIN* (i.e., we don’t use the embeddings of the plaintext images). We use CIFAR10 as our dataset, a ResNet101 architecture (trained on ImageNet) as our Encoder, and VGG16 (trained on CIFAR100) as our CMLS.

Our results are presented in Table 1. While the results are lower than those obtained for plaintext images (17% and 64.5% compared to 93.3% and 99.8%, respectively), it is clear that the CMLS's output is useful. In the remainder of this section, we analyze the output vectors' contribution when combined with the Encoder's output.

Use-Case 1: Same Labels in Confidential Data and Cloud. In this use-case, we train the CMLS on images with the same labels as confidential data (no overlap between the two sets of images). We conducted three sets of experiment, for ImageNet, CIFAR100, and CIFAR10 (for full experimental setup, see Section 4.1). Our results are presented in Table 2. We present the IIN's performance with respect to the ground truth and to the CMLS's classifications of the unencrypted images. As an upper bound, we also present the performance of the cloud model on the unencrypted images.

The results clearly show that EPKD is capable of inferring both the ground truth labels and the cloud's classifications. It is clear, however, that there is a decrease in performance. An important observation is that this decrease in performance is smaller when the *number of labels in the confidential data is small*. The reason for this phenomenon is simple: as the *labels ratio* $\frac{|L_{conf}|}{|L_{cloud}|}$ becomes smaller, each confidential data label can be expressed in a more nuanced way, by a larger number of cloud labels. Next we analyze the label ratio.

Use-Case 2: Analyzing the effect of the labels ratio. Based on use-case 1, we now revisit our ImageNet experiments (in which EPKD had the largest gap with the original cloud performance). We used the same settings as in use-case 1, but the confidential data contained only a subset of 10/100 labels instead of the original 1,000. For each of these two settings—10 and 100—we performed five experiments, with the labels of each randomly selected. We used a 70%/30% train/test split with 50 images per label, resulting in 350/3,500 images for the training of the IIN and 150/1,500 images for evaluation.

The results of our evaluation are presented in Table 3. It is clear that our performance on ImageNet improves as the number of labels decreases, with EPKD's top-1 accuracy rising from 67.9% to 95.5%, and its top-5 accuracy rising from 87.6% to 99.9%. The improvement in performance shows that we were correct in our hypothesis that $\frac{|L_{conf}|}{|L_{cloud}|}$ is a main factor in determining our method's performance.

Use-Case 3: Different labels for confidential data and cloud. We now evaluate EPKD's ability to infer labels that did not appear in the CMLS's training set. This use-case is important because it assesses whether EPKD is generic and transferable. For these experiments we used VGG16, which was trained on CIFAR 100 as our CMLS, and ResNet101, trained on ImageNet, as our Encoder.

To generate our confidential dataset, we first identified all ImageNet labels that *did not* appear in CIFAR100. Next we randomly sampled 10 labels from these labels—150 images per class, 1,500 overall. We use 90% of these images (1,350) to train our Internal Inference agent and use the remaining 10% for evaluation. This process is repeated five times, and the averaged results are presented in Table 4. For this setting, we can only present the results with respect to the original image labels, which clearly show that we

are able to reach the same level of performance for images with previously-unseen labels as we do for those on which the CMLS was trained. Additionally, we once again show that our approach is most effective when $\frac{|L_{conf}|}{|L_{cloud}|}$ is small.

Use-Case 4: Using an ensemble of Encoders. We now examine whether an ensemble of Encoders can improve the performance of our proposed approach. We use multiple pre-trained networks as our Encoders, all encrypted using the same Generative Model. The classification vectors for each individual encoding are concatenated and fed as a single input to the IIN.

We build upon the experiments presented in Table 2. When using two encoders, the network we added was ResNet50. For our third encoder we used ResNet50v2 for the ImageNet dataset and ResNet101 for CIFAR100/10. The results, presented in Table 5, show that adding encoders indeed improves EPKD's performance, although the improvement appears to be greater for the CIFAR10/CIFAR100 datasets (which concurs with our conclusion in use case 2). Using the paired-t test, we were able to determine that the two-Encoders setting significantly outperforms the single-Encoder setting with $p < 0.001$, and that three-Encoders outperform the two-Encoders with $p < 0.01$.

Use-case 5: the effect of the IIN's training size on performance. We now analyze the effect of the training set size on the performance of the IIN. While it is likely that larger training sets will lead to higher accuracy, our goal is to quantify this impact. We conducted our analysis as follows: We used ILSVRC2012's validation set, from which we randomly sampled 100 labels, to train our IIN and evaluate its performance. We used Xception as our CMLS (this model was trained on ILSVRC2012's *training set*), and ResNet101 as our Encoder.

We sampled different numbers of images from each of our selected 100 labels, and used them to train the IIN. We repeat this experiment five times and report the average results, which are presented in Figure 2. It is clear that an increase in the training set significantly improves the performance of our approach, with top-1 accuracy improving by 6.8% and top-5 accuracy improving by 2.1%. It is also clear that the improvement in EPKD's performance begins to plateau at around 30 images per label, which achieved an accuracy of 87.7% compared to the 88.5% for 45 images, indicating that a relatively small number of training samples is needed for maximal performance.

The small number of samples that is needed to train our IIN is a crucial aspect of our proposed approach. Given that each key (i.e., the randomly-initialized Generator weights) can only be used a limited number of times before an adversary can reconstruct the original images (see Sections 5.1 and 5.2 for analysis), requiring a limited number of images to train the IIN means that each key can be used for a longer period of time.

5 THE ROBUSTNESS OF THE PROPOSED APPROACH

We argue about the robustness of our proposed solution. Recall the model and the properties we require as given in Section 3.2. As we have already mentioned there, our solution proposes a middle-ground between provably secure systems (such as ones based on

Dataset	Cloud Architecture	Encoder Architecture	Internal Inference				Cloud Performance	
			Real Labels		Cloud Class.		Un-encrypted images	
			Top-1	Top-5	Top-1	Top-5	Top-1	Top-5
ImageNet	Xception	ResNet101	67.9% (0.007)	87.6% (0.006)	69.3% (0.008)	88.8% (0.004)	77.7% (0.005)	93.8% (0.004)
CIFAR100	VGG16	ResNet50V2	65.6% (0.017)	88.2% (0.015)	59.1% (0.014)	81.8% (0.016)	69.5% (0.011)	90.1% (0.004)
CIFAR10	VGG16	ResNet50V2	87.6% (0.011)	99.3% (0.002)	86% (0.012)	98.9% (0.003)	93.3% (0.009)	99.8% (0.002)

Table 2: Evaluation results for Use Case 1. Note that we compare our model’s classifications both to the classifications produced by the CMLS, as well as the ground truth.

Number of labels	Top-1	Top-5
1000	67.9% (0.007)	87.6% (0.006)
100	87.2% (0.009)	96.7% (0.006)
10	95.5% (0.019)	99.9% (0.003)

Table 3: Use Case 2: IIN’s accuracy compared to “ground truth”

Metric	Internal Inference to Real Labels
Top-1	97% (0.02)
Top-5	99.9% (0.003)

Table 4: Use Case 3: results on “ground truth”

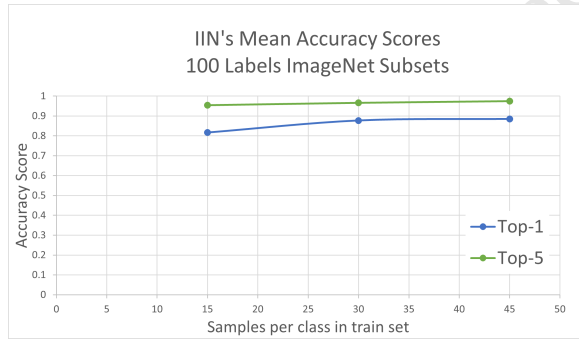


Figure 2: The top-1 and top-5 accuracy of EPKD on a randomly-sampled subset of ImageNet, consisting of 100 labels. The results are presented as a function of the number of images per label that were used in EPKD’s training set.

fully-homomorphic encryption) that are somewhat inefficient and require specialized CMLS design, and ones that are efficient but do not seem to provide any security. Specifically, while we are unable to formally prove that our system provides security by reducing from a well-known hardness assumption, we argue that it is secure with respect to a class of all known attacks for similar systems. We also provide various (experimental) statistics providing evidence that naive indistinguishability attacks cannot break our system. In particular, to attack our system, radically new ideas are needed.

Before delving into the details, let us also mention that in a real-life deployed system, the CMLS has very little (or even no) control over the inputs or the internal state of the system. However, since this is a robustness section, we often give the CMLS more power than it should have; this only makes our claims stronger. In particular, we want to emphasize three aspects of EPKD that make it robust against attacks used in prior works. First, only the encrypted images, without labels, leave the organizational network. This is a key difference from instance encoding (Section 2.2). Secondly, we use a “vanilla” deconvolutional network without the image-patches or sign-flipping used in previous solutions. This practice limits attackers’ options, as discussed in Section 2.2. Thirdly, the training of the IIN requires a small number of samples (Section 4.2, Use-case 5), meaning that keys can be changes frequently and at a very low cost (allowing a complete refresh of the internal state). To ensure that our analysis is reproducible, we share all images (plaintext and encrypted) used in the experimetns of this section².

Types of attacks. We consider three types of attacks against our system: (1) Distinguish a pair of encoded images (Section 5.1) (2) Recover the key used for the encryption mechanism, i.e., the deconvolutional network (Section 5.2), and (3) map the CMLS’s output vector to the “real” output vector (Section 5.3). A successful execution of any of these attacks is sufficient to break our security definition (Definition 3.1).

5.1 Attacks on the Encrypted Images

Figure 3 presents a pair of images—original and encrypted. It is quite clear that there is nothing that a human can derive from this image in its encrypted form. While this statement is subjective to some degree, the authors reviewed multiple images generated by multiple embedding/deconvolution combinations, and were never able to identify anything meaningful in any image.

Preliminary analysis: the efficacy of brute-force attacks. Let us analyze the less-challenging scenario, where we only encrypt the plaintext image, without fusing it with n other random images. Even in this scenario, a brute force search is impractical due to the large number of options the attacker needs to explore. Case in point: even if we take an image matrix x of size 2×2 and a kernel matrix y of size 2×2 , with the values in each matrix being integers in the range $1 \dots 2^k$ (that is, disregarding negative values

²https://osf.io/p2qt4/?view_only=5da7abe6d6a04e8b9703985f5b0f6683

datasets/ #encoders	Internal Inference												Cloud Performance	
	Real Labels						Cloud Class.						Un-encrypted images	
	1	Top-1 2	Top-1 3	Top-5 1	Top-5 2	Top-5 3	Top-1 1	Top-1 2	Top-1 3	Top-5 1	Top-5 2	Top-5 3	Top-1	Top-5
ImageNet	67.9% (0.007)	68.6% (0.002)	69.5% (0.006)	87.6% (0.006)	88.9% (0.004)	89.1% (0.006)	69.3% (0.008)	69.9% (0.004)	71.4% (0.009)	88.8% (0.004)	90.1% (0.003)	90.7% (0.005)	77.7% (0.005)	93.8% (0.004)
CIFAR100	65.6% (0.017)	67.2% (0.01)	68.4% (0.007)	88.2% (0.015)	89.8% (0.004)	90.4% (0.009)	59.1% (0.014)	61% (0.01)	62.4% (0.009)	81.8% (0.016)	84.5% (0.011)	85.7% (0.015)	69.5% (0.011)	90.1% (0.004)
CIFAR10	87.6% (0.011)	88.1% (0.014)	88.9% (0.009)	99.3% (0.002)	99.3% (0.004)	99.4% (0.001)	86% (0.0012)	86.2% (0.015)	87.4% (0.008)	98.9% (0.003)	99.2% (0.001)	99.1% (0.002)	93.3% (0.009)	99.8% (0.002)

Table 5: Use Case 4 Results – The effect of using multiple Encoders on performance.

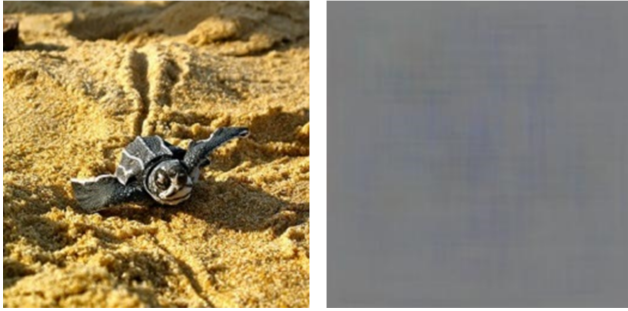


Figure 3: left) The original image. right) The same image, encrypted by EPKD using embeddings by ResNet50.

and non-natural numbers), we will have 4 cells in each matrix. The number of combinations for positive values with same result is at least $2 \cdot k$. This is the case because we can exchange the roles of the matrices, as well as perform a divide/multiply by 2 operation k times for each value in the matrices. The number of options increases exponentially both when the matrices become larger, or when we stack multiple layers. As a result, a brute-force attack quickly becomes infeasible as the analyzed architecture increases in size.

Because brute-force attacks to reconstruct our encrypted images are impractical, we focus on *distinguishing attacks* (DA), which are easier for an attacker to execute. The goal of this attack will be to determine whether two plaintext images are similar based on their encrypted forms. The ability to make this inference can be leveraged to more advanced attacks, as shown in the studies breaking NeuraCrypt and InstaHide [3, 4].

Because our approach bears some similarity to instance encoding (Section 2.2), it is important to stress the differences. While [3] demonstrates that DA can be successfully applied to break any instance encoding method, *our approach does not meet two key requirements stated in this study*. First, unlike instance encoding, our encrypted images do not contain labels. Naturally, this significantly reduces the information available to an attacker. Secondly, for the method described in [3] to be effective, the attacker needs to train an effective classifier on the analyzed data. As shown in Table 1, training a classifier on our encrypted images yields 17% accuracy

and that when we use the labels, which are not available to an attacker. For these reasons, we conclude that the attacks described in Carlini et. al's work are not applicable to EPKD. Therefore, we now evaluate two DA use-cases: one where an attacker attempts to detect samples from the same label, and one where the attacker attempts to detect the same sample, encrypted using two different keys. That is, we consider the advantage of an attacker that only "measures distances" and denote $\|\cdot\|$ the distance function; we will be more concrete below.

Use-case 1: detecting images from the same label. Consider an attack that distinguishes $\|I_{a1} - I_{a2}\|$ from $\|I_{a1} - I_b\|$, where I_{a1} and I_{a2} are images belonging to label a and I_b is an image belonging to label b where $a \neq b$, and where $\hat{I} \leftarrow \text{Enc}(\text{key}, I)$ for an image I .

We evaluate this use-case by randomly sampling 1,000 image triplets— I_{a1} , I_{a2} , and I_b —from 1000 labels of ImageNet. The nine images needed for the fusion were randomly sampled from nine other labels of the dataset (one image from each label). We test two differences: a) the distance between the pixels of each of each pair of images; b) the distance between the *embeddings of the encrypted images*, where a pre-trained ResNet101 network generates the embeddings. We use two metrics for each distance: Euclidean distance, and Cosine distance.

Using the paired t-test, we were able to determine that the differences between the images pairs *are not significant* with $p < 0.902$ (euclidean) and $p < 0.801$ (cosine). Furthermore, the differences between the embeddings of the pairs *are not significant* with $p < 0.753$ (euclidean) and $p < 0.99$ (cosine). The meaning is that an attacker cannot infer whether two encrypted images belong to the same label based on their encrypted form.

Use-case 2: detecting the same image with different keys.

Next, we consider an attacker that distinguishes $\|\text{Enc}(\text{key}, I_a) - \text{Enc}(\text{key}', I_a)\|$ from $\|\text{Enc}(\text{key}, I_a) - \text{Enc}(\text{key}, I_b)\|$, where key , key' are two randomly chosen keys. We use the same fusion method and distance metrics as in use-case 1. Furthermore, we set the bar higher by using the exact same random images for both I_a and I_b . Under any realistic scenario, this would not be the case.

Using the paired t-test, we were able to determine that the differences between the images pairs *are not significant* with $p < 0.901$ (euclidean) and $p < 0.763$ (cosine). Furthermore, the differences between the embeddings of the pairs *are not significant* with $p < 0.771$ (euclidean) and $p < 0.71$ (cosine). The meaning is that an attacker

cannot infer whether the same image was encrypted by two different keys based on its encrypted forms.

The meaning of p -values. In many experiments p -value < 0.01 is considered significant, as it means that we cannot reject the null hypothesis (that two distributions are similar) in only one out of 100 experiments. Consequently, p -value $= 0.5$ means that we cannot reject the null hypothesis in 50 out of 100 experiments, thus making the probability of observing a difference in values a coin-toss. Values near or above that threshold make distinguishing attacks impractical, due to the attacker’s inability to isolate meaningful patterns. For p -value > 0.5 , as in our case, this task becomes even more challenging.

5.2 Attacks on the Encryption Mechanism

In model extraction attacks, an attacker attempts to discover the weights of a neural network by providing it with inputs and analyzing the input-output pairs. The majority of studies in the field rely on on side-channel attacks [1, 16, 37], which assume the existence of various vulnerabilities. In our analysis we assume that no such vulnerabilities exist, and therefore turn our attention to cryptanalysis-based techniques to obtain the weights of our deconvolutional architecture.

Cryptanalysis-based model extraction approaches use techniques from the field of cipher block analysis, and are able to reduce the number of input-output pairs required to extract the model by orders of magnitude. The two leading works in this field are [5] and [23]. Because the results reported in [23] are the current SoTA, we base our analysis on the results reported in this study.

The number of input/output pairs required by [23] to extract the weights of a network is dependent on the topology of the network and its number of parameters. While the size of our deconvolutional architecture prevents us from running this attack, we can extrapolate based on the study’s reported results. These results show that for convolutional architectures, the approach requires approximately 10% of the architecture’s number of parameters. For example, a convolutional architecture of 158,088 parameters required 15,809 input/output pairs for a successful attack. Based on these results, our architecture of 90.87M parameters would require approximately 9.9M input output pairs.

We argue that even in case an attacker had the ability to query our model to carry out this attack, EPKD is robust against the aforementioned type of model extraction attacks. The *primary reason is our ability to easily change keys*. As shown in Section 4.2, we only need 3000 samples to fully train the IIN. Even if further improvements to [23] will enable it to extract the model with a number of queries that is 1% of the network’s parameters (i.e., 990,000), we will still be able to use each key for hundreds of thousands of queries, and then initialize another key. Additionally, we can increase the size of our generative model at a minimal cost (because we don’t train this network), and extend the usefulness of each key.

In addition to the primary reason mentioned above, two additional elements of our approach make model extraction attacks more difficult. First, these attacks require the ability to craft the input provided to the network, an attacker would require the ability to control the selection of the fusion images. Secondly, Both [5] and

Mean rank of top-1 label (plaintext)	3.92
Mean rank of top-1 label (encrypted)	499.8
Top-1 Mean Intersection Rate	0.1%
Top-5 Mean Intersection Rate	0.53%
Top-1 Mean Reduction	73%
Top-5 Mean Reduction	85%
Mean Entropy Increase	282.8%

Table 6: Comparison of the label intersection and relative confidence scores for plaintext and encrypted images.

[23] must know the architecture of the attacked network. While this is not currently implemented in EPKD, the introduction of even a small degree of randomness to the architecture of our Generative Model would render model extraction attacks useless. This will be easy to accomplish, given that the parameters our Generative Model are not trained, but only randomly initialized.

5.3 Attacks on the CMLS Output Vector

The goal of this analysis is to determine whether the CMLS output vector can be used to infer the labels of the plaintext images. We analyze three use-cases:

Use-case 1: inferring the labels of plaintext images from their CMLS output. The goal of this analysis is to determine the similarity of the output vectors of a plaintext image and its encrypted form. We use the Xception and ResNet101 architectures as our CMLS and Encoder models, respectively. The training and evaluation image sets were those of use-case 1 in Section 4.2.

We begin by comparing the CMLS classification vectors for each image and its encrypted counterpart. Our results, presented in Table 6, show that while the mean rank of the top-1 (i.e., correct) label for a plaintext image is 3.92 out of a thousand, the mean ranking drops to 499.8 for the encrypted image. It is clear, therefore, that the “real” labels are scattered across the entire vector. Additional analysis shows that the probabilities of any intersection of labels in the top-1 and top-5 labels is 0.1% and 0.53%, respectively. Moreover, the confidence scores assigned for each of the original top-1/5 labels have been reduced by an average of 73%-85% respectively. Finally, an analysis of the entropy of the classification vectors of plaintext and confidential images shows that the entropy of the latter is more than triple that of the former: 8.88 to 2.32. Higher entropy objectively means that less information can be extracted by an attacker.

Next, we use PCA to reduce the dimensionality of the classification vectors of original and encrypted images. We randomly choose two labels from ImageNet, each consisting of 50 samples. We present our generated 2D representations in Figure 4. It is clear that while the classification vectors for the two original groups of images are clearly separated, all the encrypted images are grouped together. Additionally, the encrypted images’ representation is clearly separated from both groups of original images. These results further show that our process is effective in obfuscating the original label of the image.

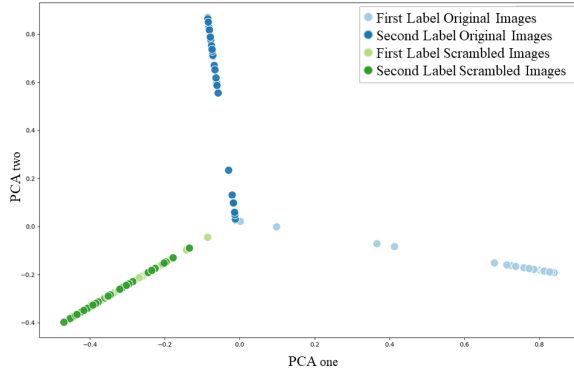


Figure 4: The classification vectors of plaintext and encrypted images of two labels, presented in a 2D space. It is clear that plaintext images are clearly separated, while the encrypted ones are mixed together.

Use-case 2: using the CMLS output to Infer that two encrypted images share a label. The goal of this analysis is to determine whether an attacker can infer that two images belong to the same label, based on the CMLS’s output vector, *when they are encrypted by the same key*. We randomly sample 1,000 images triplets from ImageNet’s validation dataset: I_{a1} and I_{a2} from label a , and I_b from label b . We then calculate the Kullback–Leibler divergence [36] distance between the following pairs: $KL(CMLS(\widehat{I}_{a1}), CMLS(\widehat{I}_{a2}))$, and $KL(CMLS(\widehat{I}_{a1}), CMLS(\widehat{I}_b))$, where CMLS is the classification function of the CMLS (i.e., the production of the classification vector), and $Enc(key, I) \leftarrow Enc(key, I)$ is the generation of the encrypted image using key key.

Using the paired t-test, we were able to determine that the differences between the pairs *are not significant* with $p < 0.913$. The meaning is that an attacker cannot infer whether two encrypted images belong to the same label based on their output CMLS output vectors.

Use-case 3: using the CMLS output to Infer that the same image is encrypted using multiple keys. The goal of this analysis is to determine whether an attacker can identify that the same image was sent twice to the CMLS, each time using a different encryption key. To analyze this question, we randomly sampled 1000 same-label pairs of images from the ImageNet dataset— I_{a1} and I_{a2} . We then encrypted I_{a1} using two encryption keys—key and key’, and calculated the KL divergence of the following pairs: $KL(CMLS(\widehat{I}_{a1}), CMLS(\widehat{I}'_{a1}))$ and $KL(CMLS(\widehat{I}_{a1}), CMLS(\widehat{I}'_{a2}))$, where for an image I , $\widehat{I} \leftarrow Enc(key, I)$ and $\widehat{I}' \leftarrow Enc(key', I)$.

Using the paired t-test, we were able to determine that the differences between the pairs *are not significant* with $p < 0.678$. The meaning is that an attacker cannot infer whether the same image was encrypted using different keys. This conclusion is important, as it shows that little to no information “leaks” across different keys.

6 CONCLUSION

We present EPKD, a one-way encryption approach that we have shown to experimentally offer non-trivial security guarantees. It requires a fraction of the computation cost requires by homomorphic encryption-based solutions and yet bypasses all known attacks. Our approach takes advantage of two key characteristics of deep neural networks: *a)* their ability to identify meaningful patterns even in significantly-transformed data, and; *b)* the fact that their output vectors are highly informative and provide a great deal of insight into their decision-making process.

Through extensive evaluation, we demonstrate that EPKD can achieve levels of performance that are almost identical to those achieved by sending the plaintext images to the CMLS. We also conduct an extensive analysis of multiple types of possible attacks against our model, and show that are approach is highly robust against all of them. These attacks include distinguishing attacks, and model extraction attacks.

For future work, we intend to explore the application of our approach both to additional domains, namely tabular data and text. We will also investigate extending our approach to additional tasks within the domain of images, with object detection and federated learning being two options.

REFERENCES

- [1] Lejla Batina, Shivam Bhasin, Dirmanto Jap, and Stjepan Picek. 2019. {CSI} {NN}: Reverse engineering of neural network architectures through electromagnetic side channel. In *28th USENIX Security Symposium (USENIX Security 19)*. 515–532.
- [2] Battista Biggio, Igino Corona, Davide Maiorca, Blaine Nelson, Nedin Šrđić, Pavel Laskov, Giorgio Giacinto, and Fabio Roli. 2013. Evasion attacks against machine learning at test time. In *Joint European conference on machine learning and knowledge discovery in databases*. Springer, 387–402.
- [3] Nicholas Carlini, Samuel Deng, Sanjam Garg, Somesh Jha, Saeed Mahloujifar, Mohammad Mahmoody, Abhradeep Thakurta, and Florian Tramèr. 2021. Is private learning possible with instance encoding?. In *2021 IEEE Symposium on Security and Privacy (SP)*. IEEE, 410–427.
- [4] Nicholas Carlini, Sanjam Garg, Somesh Jha, Saeed Mahloujifar, Mohammad Mahmoody, and Florian Tramer. 2021. NeuraCrypt is not private. *arXiv preprint arXiv:2108.07256* (2021).
- [5] Nicholas Carlini, Matthew Jagielski, and Ilya Mironov. 2020. Cryptanalytic extraction of neural network models. In *Annual International Cryptology Conference*. Springer, 189–218.
- [6] Nicholas Carlini, Chang Liu, Úlfar Erlingsson, Jernej Kos, and Dawn Song. 2019. The secret sharer: Evaluating and testing unintended memorization in neural networks. In *28th USENIX Security Symposium (USENIX Security 19)*. 267–284.
- [7] Sitan Chen, Adam Klivans, and Raghu Meka. 2021. Efficiently Learning One Hidden Layer ReLU Networks From Queries. *Advances in Neural Information Processing Systems* 34 (2021).
- [8] François Chollet. 2017. Xception: Deep learning with depthwise separable convolutions. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 1251–1258.
- [9] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. 2009. Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*. Ieee, 248–255.
- [10] Matt Fredrikson, Somesh Jha, and Thomas Ristenpart. 2015. Model inversion attacks that exploit confidence information and basic countermeasures. In *Proceedings of the 22nd ACM SIGSAC conference on computer and communications security*. 1322–1333.
- [11] Ran Gilad-Bachrach, Nathan Dowlin, Kim Laine, Kristin Lauter, Michael Naehrig, and John Wernsing. 2016. Cryptonets: Applying neural networks to encrypted data with high throughput and accuracy. In *International conference on machine learning*. PMLR, 201–210.
- [12] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 770–778.
- [13] Ehsan Hesamifard, Hassan Takabi, Mehdi Ghasemi, and Catherine Jones. 2017. Privacy-preserving machine learning in cloud. In *Proceedings of the 2017 on cloud computing security workshop*. 39–43.

- [14] Ehsan Hesamifard, Hassan Takabi, Mehdi Ghasemi, and Rebecca N Wright. 2018. Privacy-preserving machine learning as a service. *Proceedings on Privacy Enhancing Technologies* 2018, 3 (2018), 123–142.
- [15] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. 2015. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531* (2015).
- [16] Weizhe Hua, Zhiru Zhang, and G Edward Suh. 2018. Reverse engineering convolutional neural networks through side-channel information leaks. In *2018 55th ACM/ESDA/IEEE Design Automation Conference (DAC)*. IEEE, 1–6.
- [17] Yangsibo Huang, Zhao Song, Kai Li, and Sanjeev Arora. 2020. Instahide: Instance-hiding schemes for private distributed learning. In *International conference on machine learning*. PMLR, 4507–4518.
- [18] Tyler Hunt, Congzheng Song, Reza Shokri, Vitaly Shmatikov, and Emmett Witchel. 2018. Chiron: Privacy-preserving machine learning as a service. *arXiv preprint arXiv:1803.05961* (2018).
- [19] Nick Hynes, Raymond Cheng, and Dawn Song. 2018. Efficient deep learning on multi-source private data. *arXiv preprint arXiv:1807.06689* (2018).
- [20] Matthew Jagielski, Nicholas Carlini, David Berthelot, Alex Kurakin, and Nicolas Papernot. 2020. High accuracy and high fidelity extraction of neural networks. In *29th USENIX Security Symposium (USENIX Security 20)*. 1345–1362.
- [21] Guy Katz, Clark Barrett, David L Dill, Kyle Julian, and Mykel J Kochenderfer. 2017. Reluplex: An efficient SMT solver for verifying deep neural networks. In *International conference on computer aided verification*. Springer, 97–117.
- [22] Alex Krizhevsky, Geoffrey Hinton, et al. 2009. Learning multiple layers of features from tiny images. (2009).
- [23] Ryan Lehmkuhl, Pratyush Mishra, Akshayaram Srinivasan, and Raluca Ada Popa. 2021. Muse: Secure inference resilient to malicious clients. In *30th USENIX Security Symposium (USENIX Security 21)*. 2201–2218.
- [24] Ping Li, Jin Li, Zhengan Huang, Chong-Zhi Gao, Wen-Bin Chen, and Kai Chen. 2018. Privacy-preserving outsourced classification in cloud computing. *Cluster Computing* 21, 1 (2018), 277–286.
- [25] Ping Li, Jin Li, Zhengan Huang, Tong Li, Chong-Zhi Gao, Siu-Ming Yiu, and Kai Chen. 2017. Multi-key privacy-preserving deep learning in cloud computing. *Future Generation Computer Systems* 74 (2017), 76–85.
- [26] Ping Li, Tong Li, Heng Ye, Jin Li, Xiaofeng Chen, and Yang Xiang. 2018. Privacy-preserving machine learning with multiple data providers. *Future Generation Computer Systems* 87 (2018), 341–350.
- [27] Evgeny Milanov. 2009. The RSA algorithm. *RSA laboratories* (2009), 1–11.
- [28] Alec Radford, Luke Metz, and Soumith Chintala. 2015. Unsupervised representation learning with deep convolutional generative adversarial networks. *arXiv preprint arXiv:1511.06434* (2015).
- [29] Mohammad Rahimzadeh and Abolfazl Attar. 2020. A modified deep convolutional neural network for detecting COVID-19 and pneumonia from chest X-ray images based on the concatenation of Xception and ResNet50V2. *Informatics in Medicine Unlocked* 19 (2020), 100360.
- [30] David Rolnick and Konrad Kording. 2020. Reverse-engineering deep relu networks. In *International Conference on Machine Learning*. PMLR, 8178–8187.
- [31] Reza Shokri, Marco Stronati, Congzheng Song, and Vitaly Shmatikov. 2017. Membership inference attacks against machine learning models. In *2017 IEEE Symposium on Security and Privacy (SP)*. IEEE, 3–18.
- [32] Karen Simonyan and Andrew Zisserman. 2014. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556* (2014).
- [33] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. 2016. Rethinking the inception architecture for computer vision. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2818–2826.
- [34] Christian Szegedy, Wojciech Zaremba, Ilya Sutskever, Joan Bruna, Dumitru Erhan, Ian Goodfellow, and Rob Fergus. 2013. Intriguing properties of neural networks. *arXiv preprint arXiv:1312.6199* (2013).
- [35] Florian Tramèr and Dan Boneh. 2018. Slalom: Fast, verifiable and private execution of neural networks in trusted hardware. *arXiv preprint arXiv:1806.03287* (2018).
- [36] Tim Van Erven and Peter Harremoës. 2014. Rényi divergence and Kullback-Leibler divergence. *IEEE Transactions on Information Theory* 60, 7 (2014), 3797–3820.
- [37] Lingxiao Wei, Bo Luo, Yu Li, Yannan Liu, and Qiang Xu. 2018. I know what you see: Power side-channel attack on convolutional neural network accelerators. In *Proceedings of the 34th Annual Computer Security Applications Conference*. 393–406.
- [38] Adam Yala, Homa Esfahanizadeh, Rafael GL D’Oliveira, Ken R Duffy, Manya Ghobadi, Tommi S Jaakkola, Vinod Vaikuntanathan, Regina Barzilay, and Muriel Medard. 2021. Neuracrypt: Hiding private health data via random neural networks for public training. *arXiv preprint arXiv:2106.02484* (2021).

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009